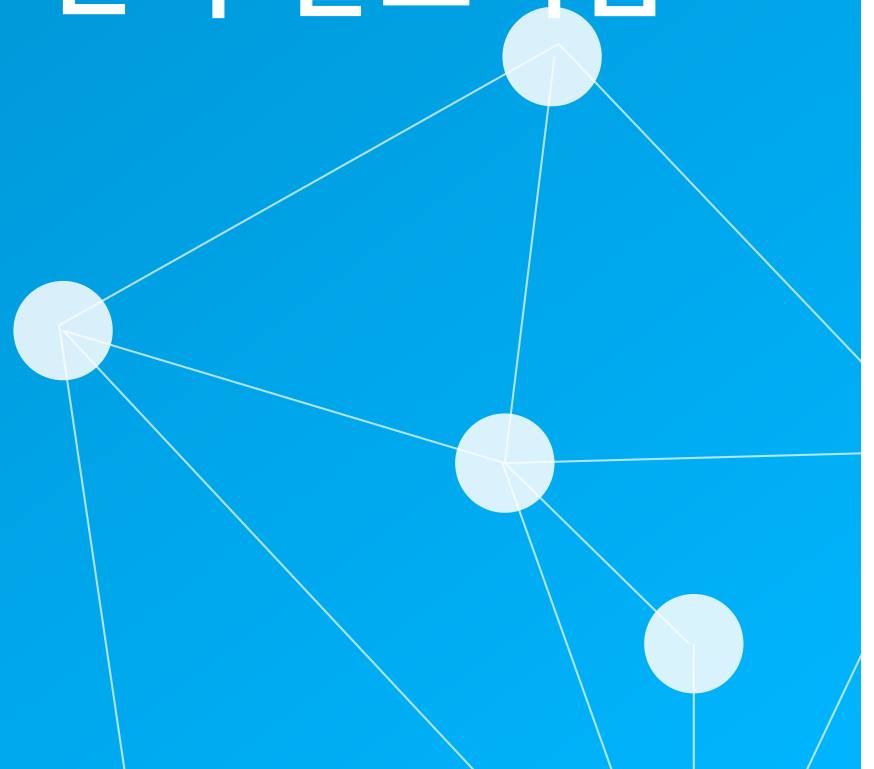


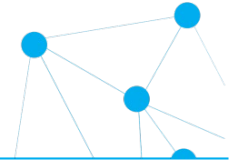
10

CHAPTER

NP-완전과 근사 알고리즘



학습 내용



10.1 문제의 분류

10.2 NP-완전 이론

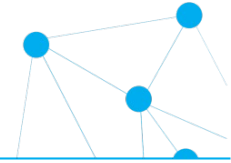
10.3 NP-완전 문제들과 근사 알고리즘

10.4 통 채우기 문제의 근사 알고리즘

10.5 정점 커버 문제의 근사 알고리즘

10.6 TSP 문제의 근사 알고리즘(심화)

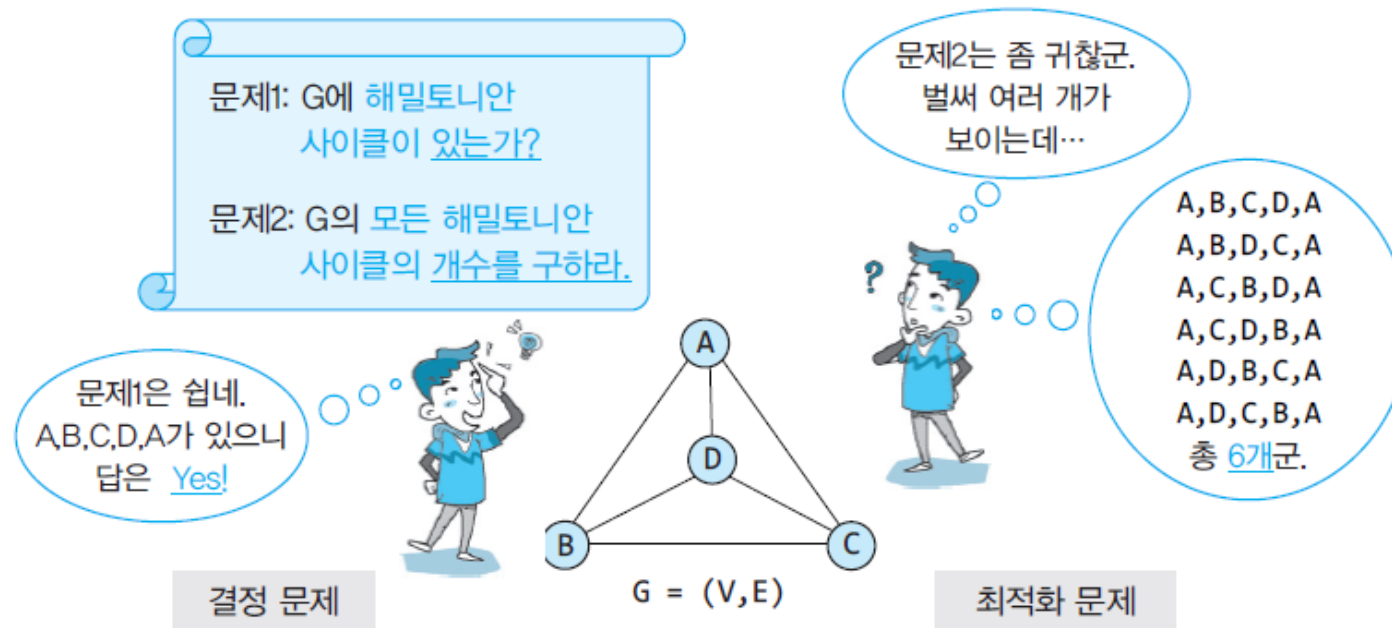
NP-완전과 근사 알고리즘



- 컴퓨터로 다룰 수 있는 문제들
 - 풀 수 있는(tractable) 문제
 - 풀 수 없는(intractable) 문제
 - 앨런 튜링이 제시한 정지 문제(halting problem)
- 풀 수 있는(tractable) 문제
 - 현실적인 시간(다항식 시간)
 - 8장까지 여러 문제들
 - 현실적인 시간에 풀 수 없는 문제
 - 9장에서 다룬 문제들
 - 지수형, 팩토리얼형
- NP-완전 문제
 - 다항식 시간에 해결할 수 없다고 추정되면서 서로 강력한 논리적인 연관성을 갖는 특별한 문제들의 그룹
- 현실적인 시간에 근사해를 구하는 방법

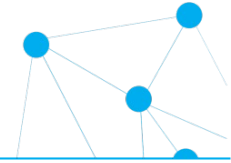
10.1 문제의 분류

- 결정 (decision) 문제와 최적화 (optimization) 문제
 - 결정 문제: 대답으로 Yes 혹은 No를 요구
 - 최적화 문제: 주어진 조건을 만족하는 최적해를 요구



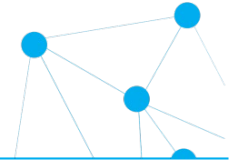
[그림 10.2] 해밀토니언 사이클 관련 결정 문제와 최적화 문제의 예

최적화 TSP → 결정 TSP



- 최적화 TSP 문제
 - 임의의 한 점에서 출발하여 다른 모든 점을 한 번씩만 방문하고 다시 시작점으로 돌아오는 최단 거리를 구하라.
- 결정 TSP 문제
 - 임의의 한 점에서 출발하여 다른 모든 점을 한 번씩만 방문하고 다시 시작점으로 돌아오는 거리 K 이하인 경로가 있는가?
- 결정 TSP 문제가 어렵다면 최적화 TSP도 어려운 문제

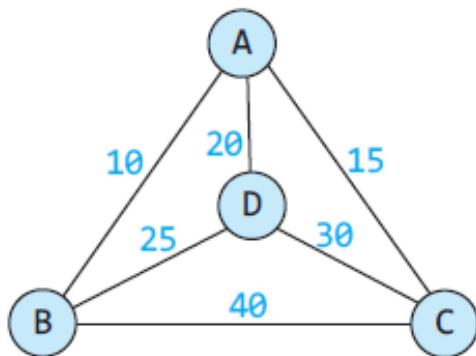
NP 문제



- 결정 문제는 P 문제와 NP 문제로 구분

- P 문제: 다항식(Polynomial) 시간에 해결할 수 있는 결정 문제
- NP 문제: 비결정론적(Nondeterministic Polynomial) 다항식 시간에 해결할 수 있는 결정 문제. 만약 결정 문제의 답이 Yes라는 근거가 주어지면 그 근거가 옳다는 것을 다항식 시간에 확인할 수 있는 문제

- 결정 TSP 문제가 NP 문제임을 보이는 과정



결정 TSP 문제: 임의의 한 점에서 출발하여 다른 모든 점을 한 번씩만 방문하고 다시 시작점으로 돌아오는 거리 90 이하인 경로가 있는가?

Yes. 근거는? → 경로 (A,B,D,C,A)

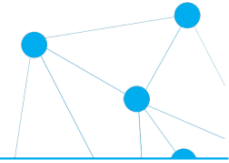
확인: (A,B,D,C,A)의 길이 < K

$$10+25+30+15 < 90$$

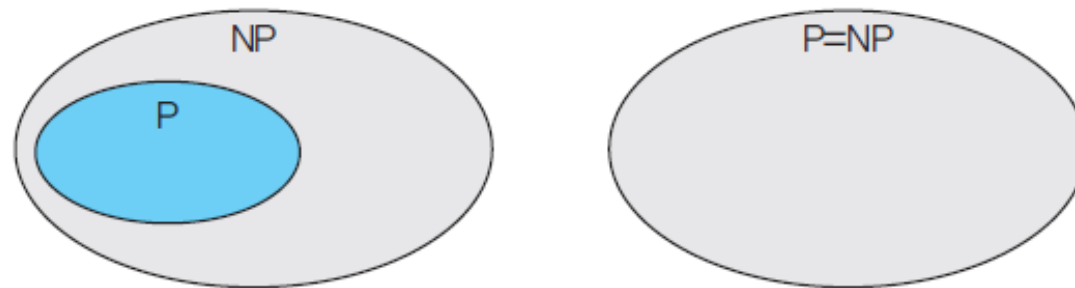
→ 답과 근거가 옳음

다항식 시간에
확인 가능

P와 NP의 두 가지 가능성

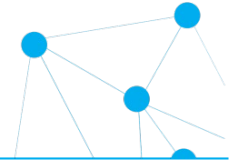


- $P=NP?$ $P \neq NP?$
 - 아무도 증명하지 못함
 - 이 문제에는 많은 현상금이 걸려있음



[그림 10.4] P와 NP의 두 가지 가능성

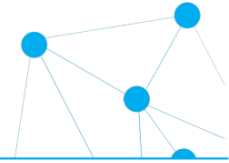
10.2 NP-완전 이론



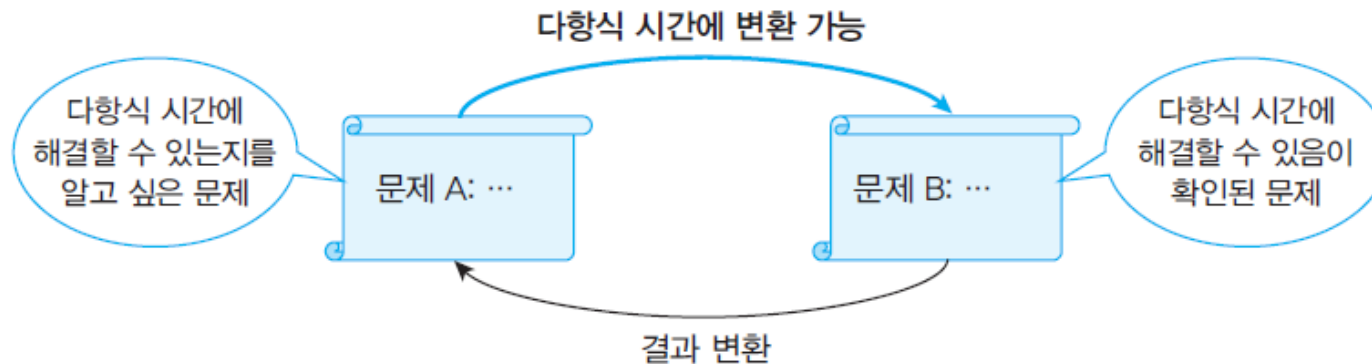
NP-완전(complete) 문제는 NP에 속하는 문제로 현재까지는 다항식 시간에 풀기는 어렵다고 판단되지만 다른 NP 문제들과 어렵다는 점에서 논리적으로 밀접하게 연결된 문제들이다. 따라서 아직 다항식 시간에 해결 가능하다는 것을 보인 문제는 없지만, 만약 하나의 NP-완전 문제라도 다항식 시간에 해결할 수 있다면 이 문제의 답을 이용해 다른 문제의 답도 구할 수 있어 모든 NP-완전 문제가 다항식 시간에 연쇄적으로 풀리게 된다.



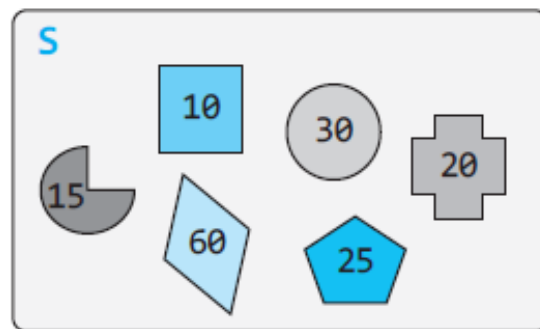
다항식 시간의 문제 변환



- 다항식 시간의 문제 변환을 통해 답을 얻는 예



- SubsetSum 과 Partition문제



SubsetSum 문제

$K=70$: {15, 30, 25}

→ 해: Yes

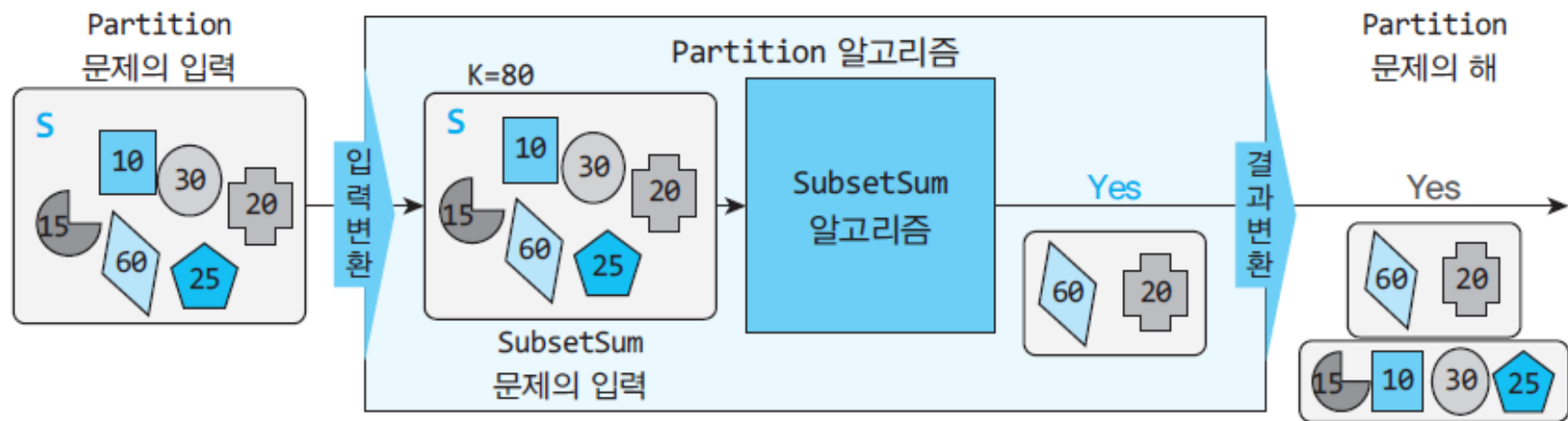
Partition 문제

분할 : {60, 20}, {15, 10, 30, 25}

→ 해: Yes

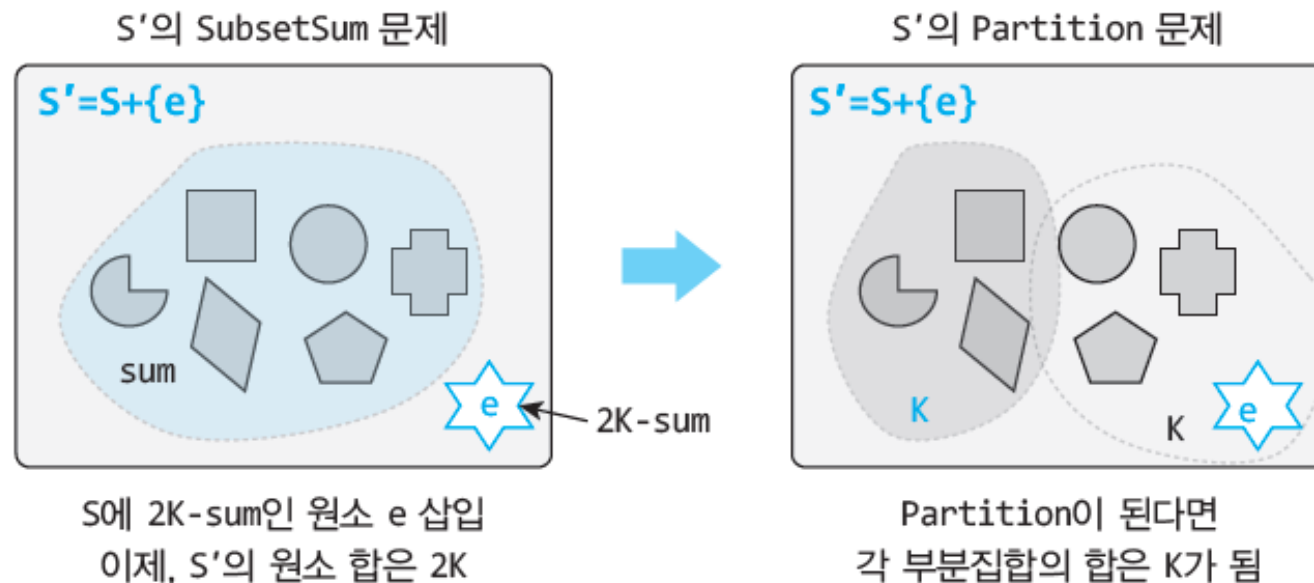
Partition 문제를 SubsetSum 문제로 변환

집합 S 의 모든 원소를 더한 값을 sum 이라 하자. Partition 문제에서 분할된 두 집합의 합은 반드시 각각 $sum/2$ 가 되어야 한다. 따라서 S 의 부분집합 중에 합이 $sum/2$ 인 집합이 있으면 분할도 가능하다. 즉, Partition 문제는 $K=sum/2$ 인 경우의 SubsetSum 문제의 결과와 정확히 일치한다.



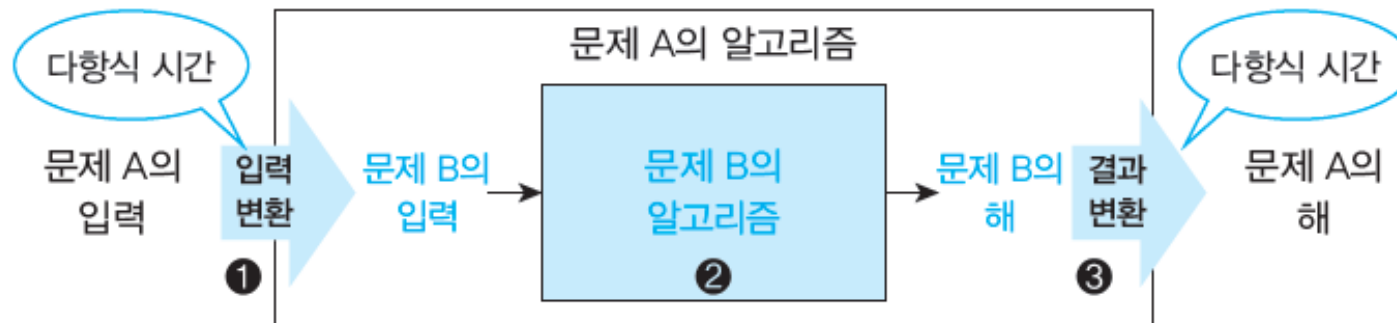
SubsetSum 문제를 Partiton 문제로 변환

아이디어는 원소의 합이 $2K$ 가 되도록 특별한 원소 e 하나를 S 에 추가하고, 이 집합에 대해 Partition 문제를 푸는 것이다. 즉, $S' = S \cup \{e\}$ 에 대한 Partition 문제의 해는 SubsetSum 문제의 해와 동일하다. e 는 $2K$ 에서 집합 S 의 원소의 합을 빼면 간단히 구할 수 있다.



NP-완전

- 변환을 통한 문제 해결에서의 전체 처리시간



정의 10.1 NP-완전(NP-complete)

어떤 결정문제 A가 다음 조건을 만족하면 이 문제는 NP-완전이다.

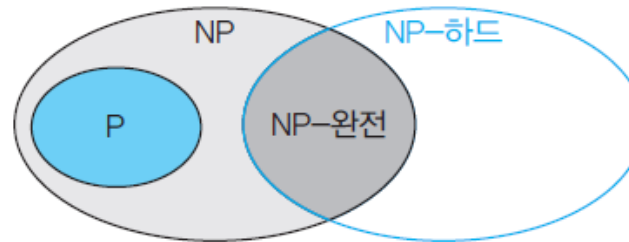
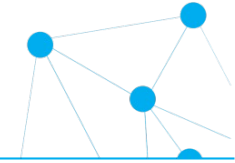
- (1) 문제 A는 NP에 속한다.
- (2) 모든 NP 문제들은 다항식 시간에 문제 A로 변환할 수 있다.

정의 10.2 NP-하드(NP-hard)

어떤 문제 A가 다음 조건을 만족하면 이 문제는 NP-하드이다.

- (1) 모든 NP 문제들은 다항식 시간에 문제 A로 변환할 수 있다.

P, NP, NP-완전, NP-하드 사이의 관계



정리 10.1

어떤 문제 A가 다음 조건을 만족하면 이 문제는 NP-하드이다.

- (1) 어떤 알려진 NP-하드 문제에 대해 이 문제를 다항식 시간에 문제 A로 변환할 수 있다.

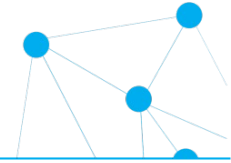
정의 10.3 NP-완전(NP-complete)

어떤 문제 A가 다음 조건을 만족하면 이 문제는 NP-완전이다.

- (1) 문제 A는 NP에 속한다.
- (2) 문제 A는 NP-하드이다.

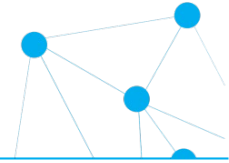
- 최초의 NP-하드 문제 → 정의 10.2에 의해 증명해야 함
 - 다행히 1971년 Cook이 일반 부울식 만족 문제(GSAT)가 NP-하드임을 증명

10.3 NP-완전 문제들과 근사 알고리즘



- 대표적인 NP-완전 문제들
 - Boolean satisfiability problem (SAT)
 - 해밀토니안 사이클과 외판원(TSP) 문제
 - 0-1 배낭 문제(01-Knapsack problem)
 - 그래프 색칠 문제(Graph coloring problem)
 - 부분집합의 합(Subset sum problem)
 - 분할 문제(Partition problem)
 - 통 채우기 문제(Bin Packing problem)
 - 정점 커버 문제(Vertex cover problem)
 - 집합 커버(set cover), 독립 집합(independent set)
 - 클리크(clique), 부분 그래프 동형(subgraph isomorphism)
 - 지배적 집합(dominating set) 등

근사 알고리즘이란?



- NP-완전 문제의 해결을 위한 접근

- 문제의 크기가 충분히 작다면 9장에서 공부한 백트래킹이나 분기 한정 기법을 잘 적용하여 현실적인 시간에 최적해를 구할 수 있다.
- 입력에 특별한 제한이 있는 경우는 다항식 시간 알고리즘을 찾을 수 있다. 예를 들어, 01-배낭 채우기 문제에서 무게가 적절한 정수 범위라면 동적 계획법을 적용해 다항식 시간에 처리할 수 있다.
- 최적해가 아니고 최적해와 비슷한 근사해가 되더라도 충분히 사용할 수 있는 경우라면 다항식 시간에 해결할 수 있는 근사 알고리즘을 찾아본다.

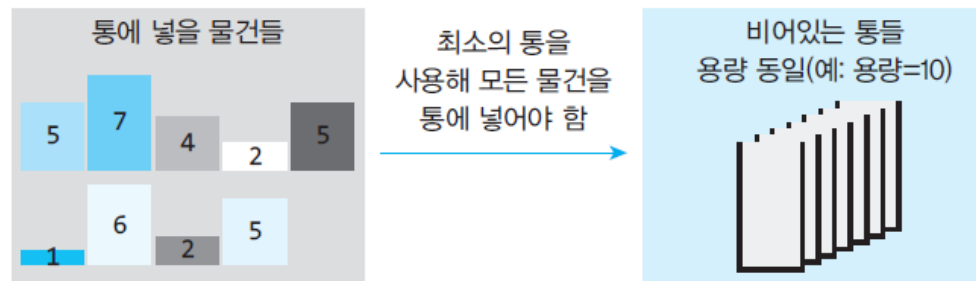
- 근사 알고리즘

- 최적해가 아니라 최적해에 가까운 근사해를 구하는 방법
- 경험적인 정보, 즉 휴리스틱(heuristic)을 사용
- 근사 비율을 제시해야 함

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n)$$

10.4 통 채우기 문제의 근사 알고리즘

n 개의 물건이 있고 용량이 C 인 균일한 크기의 통들이 있다. 가장 적은 수의 통을 사용하여 모든 물건을 통에 넣어라. 단, 하나의 물건을 여러 통에 나누어 담을 수는 없으며, 각 물건의 크기는 통의 용량 C 보다 크지 않다.

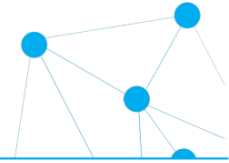


- 완전 탐색
 - 최대 가능한 경우의 수: n^n 가지의 조합
 - 근사 알고리즘
- 필요한 통의 최소 개수

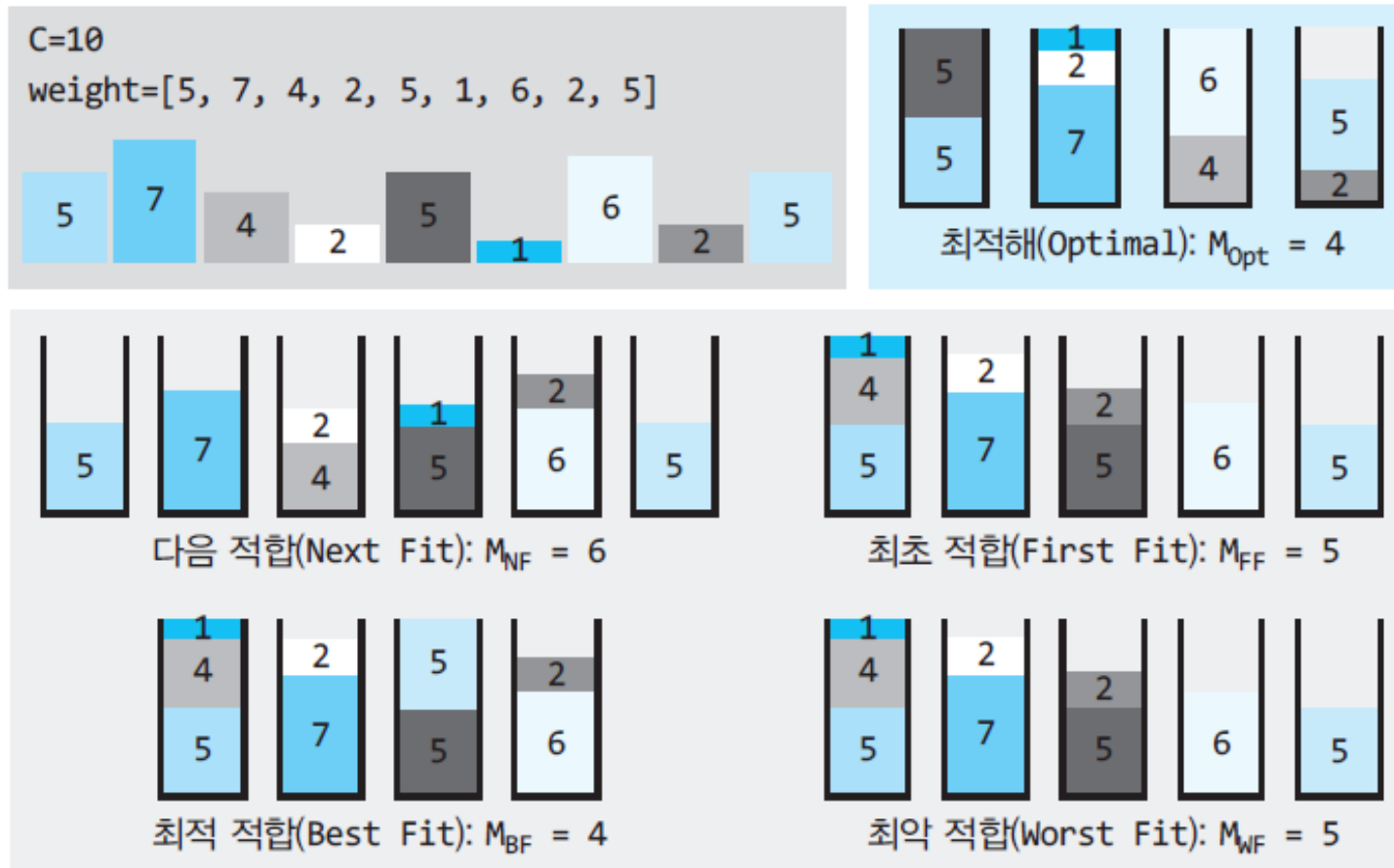
$$\text{필요한 최소 통의 수} = \left\lceil \frac{\text{모든 물건 크기의 합}}{\text{통 하나의 용량}} \right\rceil$$

- 근사 비용 계산에 사용(하한값)

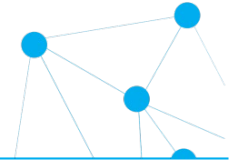
통 채우기 근사 알고리즘



- 다음/최초/최적/최악 적합



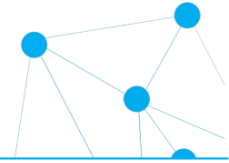
다음 적합(Next Fit)



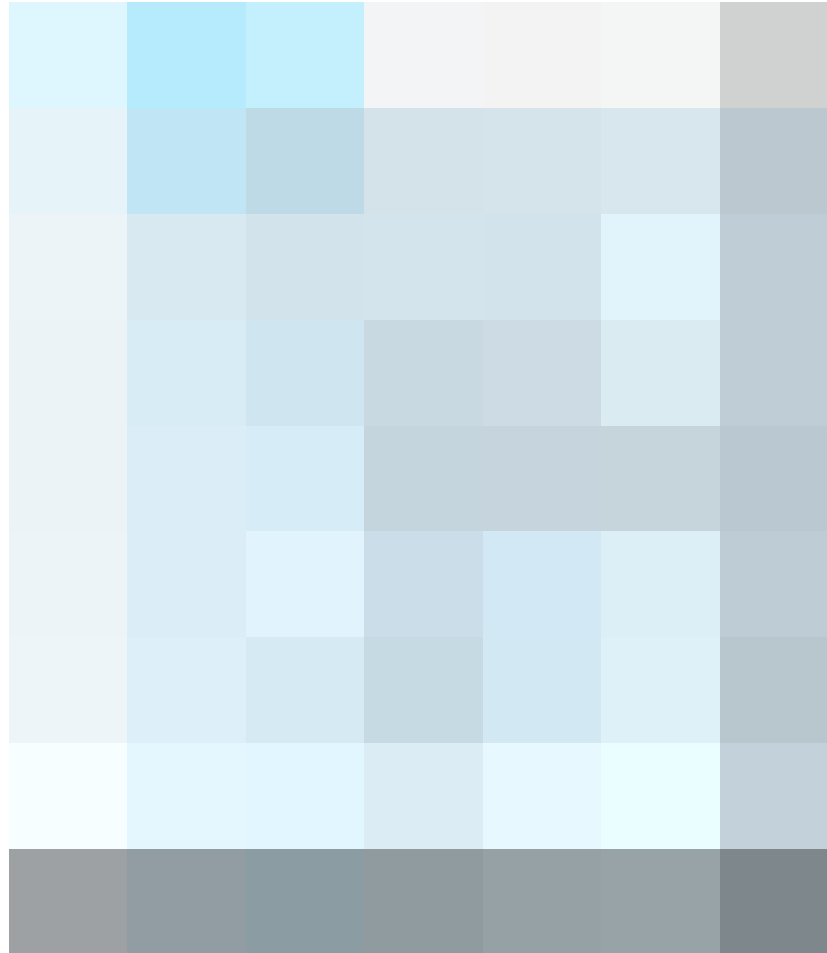
- 다음 적합 알고리즘
- 시간 복잡도
 - $O(n)$
- 근사 비율
 - 2-근사 알고리즘: why?
 - 심화학습(438쪽)



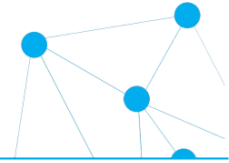
최초 적합(Next Fit)



- 최초 적합 알고리즘
- 시간 복잡도
 - $O(n^2)$
 - $O(n \log n)$ 개선 가능
- 근사 비율
 - 2-근사 알고리즘



테스트

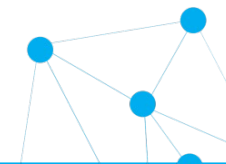


알고리즘 테스트 통 채우기 근사 알고리즘 테스트

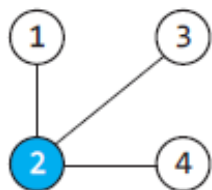
```
weight = [5, 7, 4, 2, 5, 1, 6, 2, 5]
c = 10
print(weight, c)
print("필요한 통의 수(Next Fit) : ", binPacking_NextFit(weight, c) )
print("필요한 통의 수(FirstFit) : ", binPacking_FirstFit(weight, c) )
print("필요한 통의 수(FirstFitDec) : ", binPacking_FirstFitDec(weight, c) )
```

```
C:\WINDOWS\system32\cmd.exe
[5, 7, 4, 2, 5, 1, 6, 2, 5] 10
필요한 통의 수(Next Fit) : 6
필요한 통의 수(FirstFit) : 5
필요한 통의 수(FirstFitDec) : 4
```

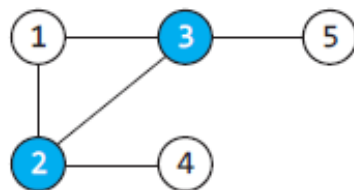
10.5 정점 커버 문제의 근사 알고리즘



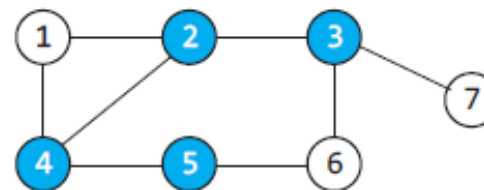
무방향 그래프 $G=(V,E)$ 에서 모든 간선 (u,v) 의 양쪽 정점 u 와 v 중에서 적어도 하나 이상의 정점을 포함하는 정점집합 중에서 최소 크기의 집합을 찾아라.



최적해 $C = \{2\}$



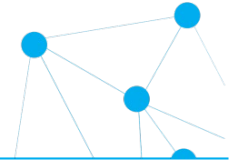
최적해 $C = \{2, 3\}$



최적해 $C = \{2, 3, 4, 5\}, \dots$

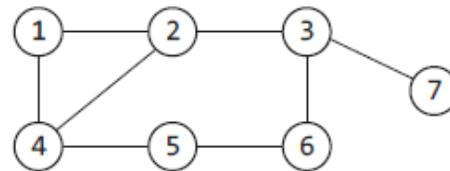
- 두 가지 전략
 - 정점 지향 전략
 - 간선 지향 전략

간선 지향 전략

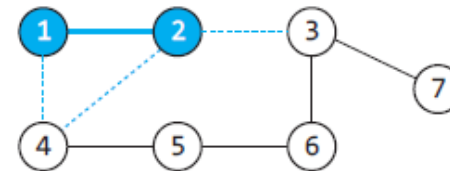


알고리즘 10.4 정점 커버 문제의 근사 알고리즘(간선 지향 전략)

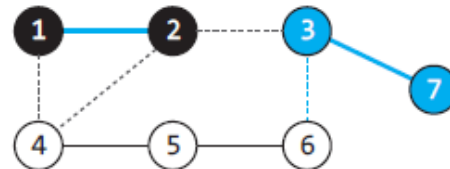
- 1: 정점 커버 집합 c 를 공집합으로 초기화한다.
- 2: 그래프의 모든 간선의 집합을 E 라 하자.
- 3: E 가 공집합이 아닐 때까지
 - 3.1: E 에서 임의의 간선 (u, v) 를 선택하고, u 와 v 를 집합 c 에 추가한다.
 - 3.2: u 와 v 에 연결된 모든 간선을 E 에서 삭제한다.
- 4: c 를 반환한다.



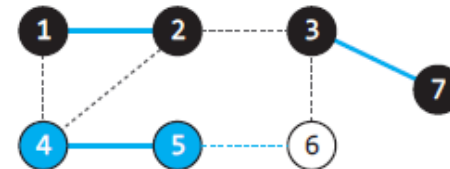
(a) 원 그래프 $G=(V,E)$



(b) 간선 $(1,2)$ 선택: $C=\{1,2\}$



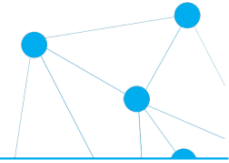
(c) 간선 $(3,7)$ 선택: $C=\{1,2,3,7\}$



(d) 간선 $(4,5)$ 선택: $C=\{1,2,3,7,4,5\}$

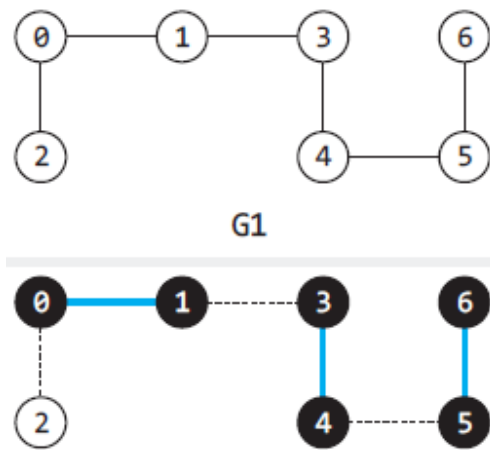
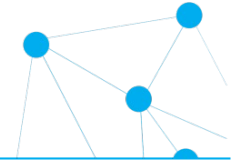
- 근사 비율
 - 2-근사 알고리즘: 심화학습(443~444쪽)

알고리즘



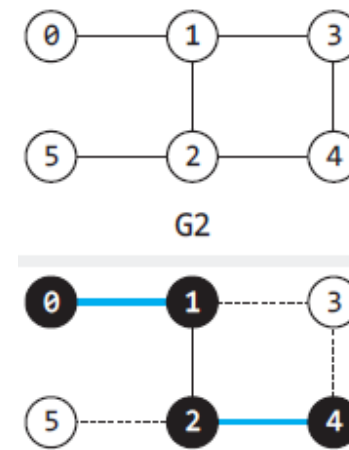
- 시간 복잡도
 - 인접 행렬: $O(n^2)$
 - 인접 리스트: $O(n + e)$

테스트



C:\WINDOWS\system32\cmd.exe

Vertex Cover(G1)= 6 [0, 1, 3, 4, 5, 6]

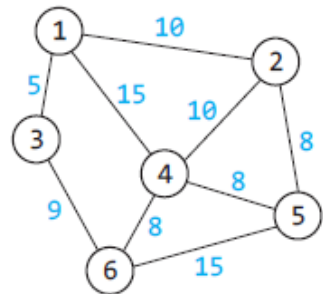


C:\WINDOWS\system32\cmd.exe

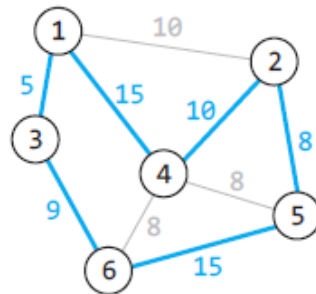
Vertex Cover(G2)= 4 [0, 1, 2, 4]

10.6 TSP 문제의 근사 알고리즘(심화)

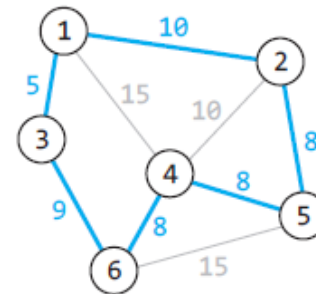
- TSP 문제의 예



그래프



경로1: 62



경로2: 48

- 최소비용 신장트리를 이용한 근사 알고리즘 → 추가조건

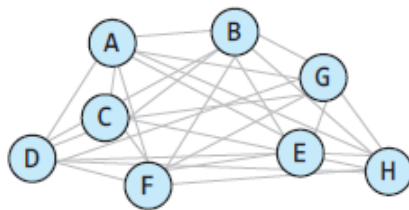
- 그래프 G 는 완전 그래프(complete graph)이다. 즉, 그래프의 모든 정점에서 다른 모든 정점으로 직접 가는 간선이 존재한다.
- 모든 간선의 가중치 $c(u,v)$ 는 음수가 아니어야 한다.
- 간선 가중치는 삼각 부등식(triangle inequality)의 원리가 적용되어야 한다. 이것은 정점 u , v , w 가 있을 때 이들 정점 사이의 간선들은 항상 다음 조건이 만족하여야 한다는 것이다.

$$c(u, w) \leq c(u, v) + c(v, w)$$

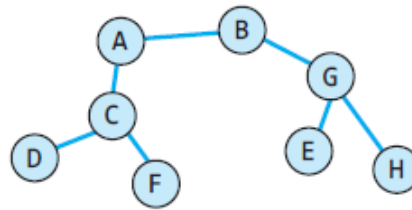
TSP 근사 알고리즘

알고리즘 10.6 TSP 문제의 근사 알고리즘

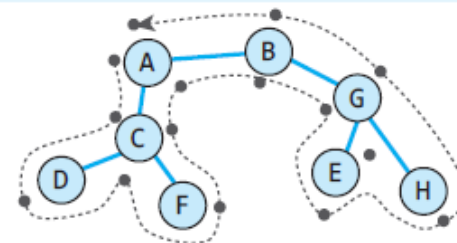
1. 입력 그래프에 대한 최소비용 신장트리 MST를 구한다.
2. MST의 임의의 노드 r 을 루트로 하여 모든 노드를 방문하고 다시 r 로 돌아오는 경로 P 를 찾는다.
3. P 에서 정점들을 순서대로 검사하여 중복되어 나타나는 모든 정점을 제거한다. 단, 시작 정점과 동일한 마지막 정점은 제거하지 않는다.
4. P 를 반환한다.



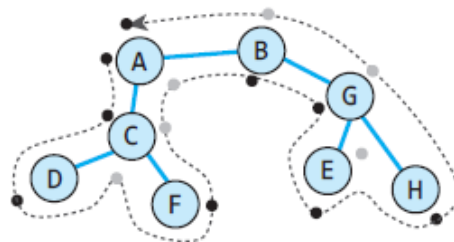
(a) $G=(V,E)$ (완전 그래프)



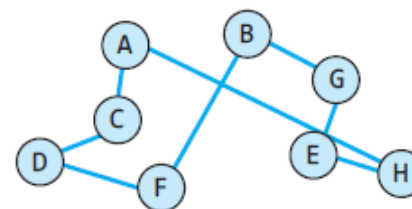
(b) G 의 최소 신장트리 T



(c) T 의 전위순회 과정에 거치는 정점:
ACDCFCABGEGHGBA



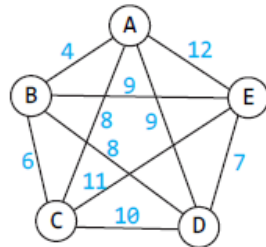
(d) 중복 정점 제거:
ACDCFCABGEGHGBA



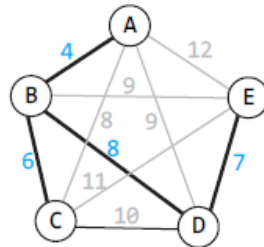
(e) 최종 근사해:
ACDFBGEHA

알고리즘 성능분석

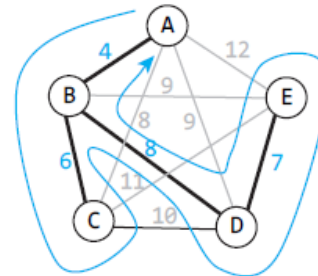
- TSP 근사 알고리즘의 근사해와 최적해의 예



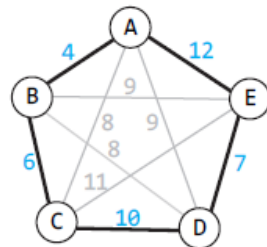
(a) $G=(V,E)$



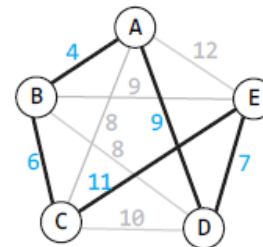
(b) G의 MST



(c) 전위순회에 거치는 정점:
ABCBDEDBA



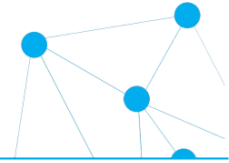
(d) 중복 정점 제거 후 최종
근사해 : ABCDEA (비용=39)

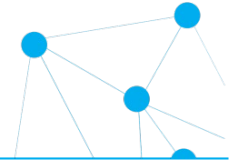


(e) 최적해: ABCEDA (비용=37)

- 2-근사 알고리즘: 심화학습(450쪽)
- 시간 복잡도
 - MST 알고리즘에 따라 결정됨

실습 과제





감사합니다!